

DCRE: Deterministic Computation with Replayable Evolution

Complete Patent Specification

Title: System and Method for Deterministic Computation with Replayable Evolution and Triangulation-Free Communication

Inventor: Bradley Clonan

Email: clonanxyz@gmail.com

Date: December 2025

Version: 1.0

Abstract

A computational system and method for achieving accountable, verifiable computation through deterministic lattice evolution with cryptographic observation. The system, termed DCRE (Deterministic Computation with Replayable Evolution), defines computation as deterministic evolution on a finite modular lattice where proof of correct execution is achieved solely through replay and fingerprint verification, requiring no trusted authorities, consensus mechanisms, or replication.

The invention further provides a novel triangulation-free communication protocol wherein messages are encoded as lattice perturbations and propagated through deterministic evolution, enabling location-independent communication that requires no routing, coordinates, or signal triangulation. Proof of transmission and reception is established through replay consistency rather than signal origin or path.

The system is formally defined as **DCRE = $(Z_m^{n \times n}, E, M, \Delta, H)$** — a closed, bounded, deterministic system where proof is the ability to replay and verify lattice evolution over time.

Unlike existing systems that rely on consensus, replication, or trusted execution, DCRE achieves proof through **reconstruction** — the ability to replay state transitions and verify fingerprint consistency.

The system is defined by the formal tuple:

DCRE = $(Z_m^{n \times n}, E, M, \Delta, H)$

Where:

- $Z_m^{n \times n}$ — Bounded modular lattice state space
 - E — Deterministic encoding function
 - M — Local block transform matrices
 - Δ — Sparse delta evolution representation
 - H — Cryptographic fingerprinting
-

Technical Field

This invention relates generally to distributed and decentralized computing systems, deterministic and bounded computation, and secure communication protocols. The invention further relates to cryptographic and non-cryptographic verification mechanisms for accountable execution, replayable state evolution, and auditable system behavior across heterogeneous computational environments.

More specifically, the invention pertains to methods and systems for location-independent communication, resilient message propagation, and coordinated computation using lattice-, grid-, or topology-based state representations. These methods enable verification, reconstruction, and validation of computational actions and communications without reliance on centralized authorities, synchronized clocks, fixed infrastructure, or continuous connectivity.

The techniques described herein are applicable to civilian, commercial, industrial, governmental, and defense systems, including but not limited to mission-critical computing, secure communications, distributed sensing, autonomous systems, cyber-physical infrastructure, contested or degraded network environments, and environments requiring deterministic behavior, auditability, fault tolerance, or operational integrity under adversarial conditions.

Background of the Invention

Limitations of Existing Systems

Current approaches to verifiable and accountable computation face fundamental limitations:

Blockchain and Distributed Ledger Technology: Requires global consensus among all participants, resulting in high latency, massive computational redundancy, and significant energy consumption. Proof of work/stake mechanisms introduce economic barriers and centralization pressures.

Conflict-free Replicated Data Types (CRDTs): Provide eventual consistency through mathematical merge operations but offer no accountability trail. Changes cannot be proven to have occurred in any particular sequence.

Trusted Execution Environments (TEEs): Depend on hardware trust assumptions that may be compromised. Security relies on manufacturer integrity and absence of hardware vulnerabilities.

Traditional Audit Logs: Susceptible to tampering, lack cryptographic binding between entries, and require trust in the logging authority.

Neural Compression Systems: While recent advances (NVIDIA Maxine, VQ-VAE, HiFiC) achieve impressive compression through shared generative models, these systems are domain-specific (typically media), approximate rather than exact, and lack verification mechanisms.

Existing Communication Protocols: Current peer-to-peer and networked communication systems rely on routing tables, address resolution, and signal triangulation to establish communication paths. These dependencies create points of failure, enable surveillance, and require coordination infrastructure.

Objects of the Invention

It is an object of this invention to provide a computational system where proof of correct execution is achieved through replay verification alone, requiring no trusted authorities.

It is a further object to provide a communication protocol that operates without routing, triangulation, or location information.

It is a further object to provide a universal framework applicable to any encodable, observable process regardless of domain.

It is a further object to provide compact proofs through sparse delta representation rather than full state history.

Summary of the Invention

The present invention provides a deterministic computational primitive defined by the tuple:

$$\text{DCRE} = (Z_m^{\{n \times n\}}, E, M, \Delta, H)$$

Where:

- $\mathbb{Z}_m^{n \times n}$ — Bounded modular lattice state space
- E — Deterministic encoding function mapping inputs to lattice states
- M — Local block transform matrices governing evolution
- Δ — Sparse delta representation of state changes
- H — Cryptographic fingerprinting functions

The system operates according to twenty axioms organized into seven groups, ensuring bounded arithmetic, deterministic evolution, replayable transitions, and verifiable observations.

A key embodiment provides triangulation-free communication wherein messages are encoded as lattice perturbations, propagated through shared deterministic evolution, and verified through replay consistency.

The system explicitly avoids consensus, leader election, quorum agreement, or majority validation as proof mechanisms. The system is designed to be universally applicable, compact in proof size, and deterministic in execution.

System Architecture Overview

In one embodiment, the invention is implemented as a system comprising:

1. A bounded arithmetic module configured to perform digit-wise arithmetic in $\mathbb{Z}_m$ without carry propagation;
2. A lattice state store maintaining a finite $n \times n$ bounded modular grid;
3. A deterministic encoder mapping arbitrary input data to an initial lattice state;
4. A lattice transform engine applying local block transformations governed by fixed matrices;
5. A delta evolution engine recording sparse state transitions between lattice epochs;
6. A delta application order is deterministic and independent of execution environment.
7. A fingerprinting module generating cryptographic fingerprints of lattice states and deltas;
8. A replay verification engine reconstructing lattice states from delta sequences to verify execution;
9. A communication channel manager maintaining one or more deterministic evolution lanes.

Each component may be implemented in software, hardware, firmware, or any combination thereof.

Detailed Description of the Invention

Part 1: Formal Axiomatic Specification

Definitions

Let:

- $m \in \mathbb{N}$, $m \geq 2$ be a fixed modulus
- $n \in \mathbb{N}$, $n \geq 2$ be a fixed grid size
- $Z_m = \{0, 1, \dots, m-1\}$

The state space is defined as:

$$G = Z_m^{n \times n}$$

Each system state is a grid $G \in G$.

Axiom Group I — Bounded Arithmetic

Axiom 1 (Boundedness): All numeric values in the system exist exclusively in Z_m . No operation may produce a value outside this set. There is no overflow, underflow, or unbounded magnitude.

Axiom 2 (Digit-wise Computation): All numbers are treated digit-wise, not as scalar magnitudes. Multi-digit values are decomposed into independent residues with no carry propagation between digits. Arithmetic is local to each digit position.

Example: For base-10, the value 13 decomposes to (1, 3). Operations proceed independently on each digit without carrying.

This property distinguishes DCRE arithmetic from both standard computer arithmetic (which uses carry propagation) and modular arithmetic (which treats numbers as scalars). Digit-wise computation enables:

- Predictable, bounded behavior
- Parallel computation without inter-digit dependencies
- Natural alignment with lattice structure

Axiom 3 (Zero as First-Class): Zero is not absence. The value $0 \in Z_m$ is a stable, meaningful state that participates in transformations identically to all other values. Padding, emptiness, and null states collapse to zero deterministically.

Axiom 4 (Structural Equality): Equality is defined by structural equivalence under transform constraints, not scalar magnitude. Two values $x, y \in \mathbb{Z}_m$ may be considered equal if $x \equiv y$ under the active transform constraints.

Axiom Group II — Grid and Lattice Structure

Axiom 5 (Finite Lattice): The system state is always a finite lattice $G \in \mathcal{G}$. Grid size $n \times n$ is fixed at system definition. Grid topology is immutable. All computation is spatially addressable.

Axiom 6 (Shared Base Pattern): All actors and executions share the same grid dimensions, the same modulus m , and the same transformation rules. Differences between executions arise only from initial state, number of steps, and observation points.

Axiom 7 (Deterministic Encoding): There exists a deterministic encoding function $E: \text{Input} \rightarrow G$ such that identical inputs produce identical grids, small input changes produce localized grid differences, and no semantic interpretation is required. Encoding is purely numeric and structural.

Axiom Group III — Lattice Dynamics

Axiom 8 (Locality): All state evolution occurs through local neighborhood transforms. No operation may reference non-local grid positions.

Axiom 9 (Modular Linear Transform): State evolution is governed by fixed, small integer matrices applied to local blocks:

$$G_{t+1}[B] = M \cdot G_t[B] \pmod{m}$$

Where $B \subset G$ is a local block (e.g., 2×2), $M \in \mathbb{Z}^{k \times k}$ is a transform matrix, and $k \ll n$.

Axiom 10 (Determinism): For any initial grid G_0 , transform matrix M , and step count t , the state G_t is uniquely determined. There is no randomness, entropy, or external input during evolution.

Axiom 11 (Reversibility): If $\det(M)$ is invertible mod m , the transformation is reversible. Irreversibility is allowed but must remain deterministic.

Transform Matrix Families

In various embodiments, the transform matrix M is selected from a family of small integer matrices including, but not limited to, identity matrices, shear matrices, swap matrices, Fibonacci-derived matrices, and other invertible or non-invertible matrices operating modulo m .

Reversible transforms are those whose determinants are invertible modulo m . Irreversible transforms remain deterministic and replayable.

Axiom Group IV — Delta Evolution

Axiom 12 (Delta Representation): System evolution is represented as deltas:

$$\Delta_t = G_{\{t+1\}} - G_t$$

Only changed cells are recorded, providing sparse representation.

Axiom 13 (Replayability): Given G_0 and $\{\Delta_0, \dots, \Delta_t\}$, the system state G_t can be reconstructed exactly.

Axiom Group V — Time and Horizon

Axiom 14 (Finite State, Infinite Time): The state space G is finite, but time t is unbounded. Cycles are allowed. Recurrence is meaningful. Termination is not required.

Axiom 15 (No Numeric Explosion): Because arithmetic is bounded and local, state evolution cannot diverge. All trajectories remain within G .

Axiom Group VI — Observation and Proof

Observer and Verifier Roles An observer is any entity that records lattice states, deltas, or fingerprints without altering system state. A verifier is any entity capable of reconstructing lattice evolution from recorded data to validate a claim. Observers and verifiers may be the same or different entities and require no trusted relationship.

Axiom 16 (External Observation): Observation does not affect computation. An observer may sample the grid, deltas, and fingerprints but cannot alter state.

Axiom 17 (Fingerprinting): Any grid or delta sequence may be reduced to a cryptographic fingerprint $H(G)$, $H(\Delta)$. These fingerprints uniquely identify the state or transition history.

Axiom 18 (Proof by Replay): A claim about system behavior is valid if and only if the grid evolution can be replayed and the recomputed fingerprints match the observed ones. Proof is reconstruction, not trust.

Axiom 19 (Observation $\neq$ Truth): The system does not assert semantic truth. It asserts that a deterministic process occurred and that an observer recorded it consistently.

Axiom Group VII — Universality

Axiom 20 (Domain Independence): This primitive applies equally to AI inference, software execution, user interaction, system events, and physical observation. Any process that can be encoded, evolved, and observed can produce a receipt.

Part 2: Triangulation-Free Communication Embodiment

2.1 Overview

In a preferred embodiment, the DCRE lattice serves as a shared deterministic communication medium. Messages are encoded as initial lattice perturbations and propagated forward in time via deterministic evolution. All participants share identical lattice parameters (n , m) and transformation rules (M).

Communication does not rely on:

- Sender or receiver location
- Routing tables or path discovery
- Signal triangulation or coordinate systems
- Network topology or intermediary nodes

Instead, messages are identified by deterministic evolution and replay recognition. A receiving party reconstructs lattice states over time and detects messages through structural or cryptographic fingerprint matching. Proof of transmission and reception is established through replay consistency rather than signal origin or path.

Epoch Synchronization Model

Participants are assumed to share a common epoch progression model, which may be derived from wall-clock time, block height, step count, or any agreed deterministic counter. Perfect synchronization is not required; replay verification tolerates bounded skew so long as the claimed epoch range can be reconstructed deterministically.

2.2 Bowling Alley Architecture

The communication model employs a "bowling alley" metaphor wherein:

1. **Parallel Lanes:** Multiple independent communication channels ("lanes") operate simultaneously without interference. Each lane maintains its own lattice state and evolution history.
2. **Straight-Line Propagation:** Messages travel "straight down the lane" through deterministic evolution — no routing decisions, no path selection, no intermediary hops.
3. **No Triangulation:** Traditional communication requires triangulating positions (sender address, receiver address, routing path). DCRE communication identifies messages by their perturbation signature and evolution fingerprint, not by location.
4. **Shared Rules, Independent State:** All participants share the same evolution rules (transform matrix M , modulus m , grid size n) but each channel maintains independent state.

2.3 Message Encoding and Transmission

Step 1: Perturbation Encoding

A message M_msg is encoded as a lattice perturbation P :

$$P = \{(\text{row_i}, \text{col_i}, \delta_i)\} \text{ for } i = 1..k$$

Where each tuple specifies a cell position and delta value. The perturbation is derived deterministically from the message content:

$$\text{bytes} = \text{Encode}(M_msg)$$

$$P[i] = (Li/nJ, i \bmod n, \text{bytes}[i] \bmod m)$$

A perturbation fingerprint $H(P)$ uniquely identifies the message.

Step 2: Lattice Perturbation

The perturbation is applied to the current channel lattice state:

$$G'[\text{row_i}][\text{col_i}] = (G[\text{row_i}][\text{col_i}] + \delta_i) \bmod m$$

Step 3: Evolution

The perturbed lattice evolves forward one or more steps:

$$G_{\{t+1\}} = \text{Evolve}(G_t, M)$$

Step 4: Receipt Generation

A transmission receipt is created:

```
Receipt = {  
  
  perturbationHash: H(P),  
  
  beforeEpoch: t,  
  
  afterEpoch: t+1,  
  
  latticeFingerprint: H(G_{t+1}),  
  
  timestamp: now()  
}
```

2.4 Message Reception and Verification

Detection Method 1: Perturbation Matching

A receiver with knowledge of the expected perturbation hash can verify receipt:

1. Maintain local copy of channel lattice
2. Evolve lattice synchronously with sender
3. On each evolution, compute $H(G_t)$
4. Match against known perturbation hashes

Detection Method 2: Replay Verification

Given a claimed transmission receipt:

1. Replay channel evolution from known checkpoint
2. Verify that perturbation at claimed epoch produces claimed fingerprint
3. Verify subsequent evolution consistency

Detection Method 3: Structural Analysis

For receivers without prior knowledge of message content:

1. Monitor lattice evolution for structural anomalies
2. Perturbations create detectable patterns in evolution trajectory
3. Extract perturbation through inverse analysis (if transform is reversible)

2.5 Security Properties

Location Privacy: Participants are not identified by address, IP, or physical location. Identity (if any) is bound to perturbation signatures.

Path Independence: No routing path is established or recorded. Communication occurs through shared deterministic evolution, not signal propagation.

Replay Attack Resistance: Each transmission receipt is bound to specific epoch numbers. Replaying old perturbations produces different fingerprints due to changed lattice state.

Integrity: Message integrity is cryptographically bound to perturbation hash and lattice fingerprint. Any modification is detectable through replay verification.

2.6 Multi-Party Communication

Broadcast: A single perturbation affects all parties sharing the channel lattice. All synchronized participants detect the message simultaneously.

Selective Reception: Different receivers can filter for perturbations matching specific patterns or hash prefixes, enabling selective reception without separate channels.

Parallel Channels: Multiple independent channels can operate simultaneously, each with distinct lattice states, enabling private communication alongside broadcast.

Physical Signal Embodiments

In certain embodiments, lattice perturbations or evolution fingerprints are transmitted over physical media including radio frequency (RF), optical, wired, acoustic, or hybrid channels. The physical medium is used solely to convey perturbation data or checkpoints; communication semantics arise from deterministic lattice evolution rather than signal path, strength, or origin.

Deterministic Collision Handling and Anti-Jamming

In embodiments involving concurrent perturbations or adversarial interference, multiple inputs injected into a lattice at the same epoch produce a deterministic composite lattice state due to bounded, digit-wise arithmetic without carry propagation. Signal collisions do not result in ambiguity, loss of information, or nondeterminism.

Interference and jamming attempts manifest as structured, replayable lattice evolution rather than destructive noise. As a result, interference is observable, attributable, and verifiable through replay consistency.

Part 3: Universal Domain Applications

DCRE applies to any process that can be encoded, evolved, and observed:

3.1 AI/ML Accountability

- **Input Encoding:** Model inputs encoded to lattice
- **Inference as Evolution:** Computation represented as lattice transformation
- **Output Verification:** Receipts prove specific inputs produced specific outputs
- **Audit Trail:** Complete history of all inferences with cryptographic binding

3.2 Software Execution Logging

- **State Encoding:** Program state transitions encoded as deltas
- **Execution Path:** Evolution history captures execution sequence
- **Replay Debugging:** Any historical state reconstructible from deltas
- **Tamper Evidence:** Modified logs produce invalid fingerprints

3.3 Financial Transactions

- **Transaction Encoding:** Transfers, balances, and events encoded to lattice
- **Settlement Proof:** Receipts prove transaction execution
- **Audit Compliance:** Complete, verifiable audit trail
- **No Double-Spend:** Deterministic evolution prevents state conflicts

3.4 Healthcare Records

- **Patient State:** Medical history as lattice evolution
- **Treatment Recording:** Interventions as perturbations
- **Privacy Preservation:** Fingerprints enable verification without revealing content
- **Chain of Custody:** Receipts prove record access and modification

3.5 Supply Chain Tracking

- **Asset Encoding:** Physical items represented in lattice
- **Transfer Events:** Custody changes as perturbations
- **Provenance Proof:** Complete history from origin to current location
- **Counterfeit Detection:** Invalid evolution signatures indicate tampering

3.6 IoT and Sensor Networks

- **Reading Encoding:** Sensor values as lattice perturbations
- **Temporal Sequence:** Evolution captures measurement timeline
- **Calibration Verification:** Consistent evolution proves sensor integrity
- **Data Integrity:** Tampered readings produce invalid fingerprints

3.7 Legal and Contract Execution

- **Contract State:** Agreement terms encoded to lattice
- **Execution Events:** Performance as perturbations
- **Signature Verification:** Digital signatures bound to lattice state
- **Dispute Resolution:** Replay provides authoritative history

3.8 Voting and Governance

- **Ballot Encoding:** Votes as lattice perturbations
- **Tally Evolution:** Aggregation through deterministic transformation
- **Verifiability:** Voters can verify inclusion without revealing choice
- **Auditability:** Complete, replayable election history

3.9 Scientific Research

- **Experiment Logging:** Observations encoded as perturbations
- **Reproducibility:** Identical encoding produces identical lattice
- **Publication Proof:** Receipts establish priority dates
- **Data Integrity:** Tampered results produce invalid fingerprints

3.10 Communication (Bowling Alley)

- **Message Encoding:** Content as lattice perturbation
- **Transmission:** Evolution propagates message
- **Reception:** Fingerprint matching detects arrival
- **Verification:** Replay proves transmission occurred

3.11 Identity and Credentials

- **Credential Issuance:** Attributes encoded as perturbations
- **Verification:** Fingerprint matching confirms validity
- **Selective Disclosure:** Partial lattice revelation
- **Revocation:** New perturbations invalidate old credentials

3.12 Gaming and Virtual Assets

- **Game State:** World state as lattice
 - **Player Actions:** Inputs as perturbations
 - **Asset Ownership:** Items bound to lattice positions
 - **Fair Play:** Deterministic evolution prevents cheating
-

Part 4: Technical Implementation

4.1 Bounded Arithmetic Implementation

$\text{mod}(\text{value}, m) = ((\text{value} \% m) + m) \% m$

`toDigits(value, base, numDigits):`

`digits = []`

`v = |floor(value)|`

`for i = 0 to numDigits-1:`

`digits[i] = v % base`

`v = floor(v / base)`

`return digits`

`addDigitWise(a, b, base, numDigits):`

`aDigits = toDigits(a, base, numDigits)`

`bDigits = toDigits(b, base, numDigits)`

`return map(i => (aDigits[i] + bDigits[i]) % base)`

4.2 Lattice Encoding

`encode(input, n, m):`

`lattice = new Grid(n, m)`

`bytes = toBytes(input)`

`for i = 0 to n*n - 1:`

`row = floor(i / n)`

`col = i % n`

`lattice[row][col] = (i < len(bytes)) ? bytes[i] % m : 0`

```

// Mixing pass for distribution

for row = 0 to n-1:

    for col = 0 to n-1:

        neighbors = [lattice[row][col],

                     lattice[row][(col+1)%n],

                     lattice[(row+1)%n][col]]

        lattice[row][col] = mod(neighbors[0] + digitalRoot(neighbors[1] + neighbors[2], m), m)

    return lattice

```

4.3 Block Transform Evolution

applyBlockTransform(lattice, M, row, col):

```

m = lattice.modulus

// Get 2x2 block

a = lattice[row][col]

b = lattice[row][col+1]

c = lattice[row+1][col]

d = lattice[row+1][col+1]

// Apply M to column vectors

newA = mod(M[0][0]*a + M[0][1]*c, m)

```

```
newC = mod(M[1][0]*a + M[1][1]*c, m)
```

```
newB = mod(M[0][0]*b + M[0][1]*d, m)
```

```
newD = mod(M[1][0]*b + M[1][1]*d, m)
```

```
// Write back
```

```
lattice[row][col] = newA
```

```
lattice[row][col+1] = newB
```

```
lattice[row+1][col] = newC
```

```
lattice[row+1][col+1] = newD
```

```
evolve(lattice, M):
```

```
    result = clone(lattice)
```

```
    n = lattice.size
```

```
// Checkerboard pattern for deterministic ordering
```

```
for phase = 0 to 1:
```

```
    for row = 0 to n-2 step 2:
```

```
        for col = phase to n-2 step 2:
```

```
            applyBlockTransform(result, M, row, col)
```

```
result.epoch = lattice.epoch + 1
```

```
return result
```

4.4 Delta Computation and Replay

```
computeDelta(before, after):
```



```
changes = []
```

```
m = before.modulus
```

```
for row = 0 to before.size - 1:
```

```
    for col = 0 to before.size - 1:
```

```
        if before[row][col] != after[row][col]:
```

```
            changes.append({
```

```
                row: row,
```

```
                col: col,
```

```
                from: before[row][col],
```

```
                to: after[row][col],
```

```
                delta: mod(after[row][col] - before[row][col], m)
```

```
            })
```

```
return {
```

```
    fromEpoch: before.epoch,
```

```
    toEpoch: after.epoch,
```

```
    changes: changes
```

```
}
```

```
replay(initial, deltas):
```

```
    current = clone(initial)
```

```
    for delta in deltas:
```

```
for change in delta.changes:

    current[change.row][change.col] = change.to

current.epoch = delta.toEpoch

return current
```

4.5 Fingerprinting

fingerprint(lattice):

```
hash = 2166136261 // FNV-1a offset basis
```

```
prime = 16777619
```

```
for row = 0 to lattice.size - 1:
```

```
    for col = 0 to lattice.size - 1:
```

```
        hash = hash XOR lattice[row][col]
```

```
        hash = hash * prime
```

```
// Include metadata
```

```
hash = hash XOR lattice.size
```

```
hash = hash * prime
```

```
hash = hash XOR lattice.modulus
```

```
hash = hash * prime
```

```
hash = hash XOR lattice.epoch
```

```
hash = hash * prime
```

```
return toHex(hash, 8)
```

4.6 Proof by Replay

```
verifyByReplay(initial, deltas, claimedFingerprint, targetEpoch):
```

```
    deltasToReplay = (targetEpoch != null)
```

```
        ? filter(deltas, d => d.toEpoch <= targetEpoch)
```

```
        : deltas
```

```
    final = replay(initial, deltasToReplay)
```

```
    computed = fingerprint(final)
```

```
    return {
```

```
        valid: computed == claimedFingerprint,
```

```
        computed: computed,
```

```
        claimed: claimedFingerprint,
```

```
        stepsReplayed: len(deltasToReplay),
```

```
        finalEpoch: final.epoch
```

```
    }
```

Part 5: Example Claims and Embodiments

Independent Claims

Claim 1: A method for deterministic computation comprising:

- (a) encoding arbitrary input data to a bounded modular lattice $G \in \mathbb{Z}_m^{n \times n}$ using a deterministic encoding function E ;

- (b) evolving said lattice through local block transforms according to $G_{t+1}[B] = M \cdot G_t[B] \pmod{m}$ where B is a local block and M is a transform matrix;
- (c) recording sparse delta representations $\Delta_t = G_{t+1} - G_t$ comprising only changed cells;
- (d) generating cryptographic fingerprints $H(G)$ uniquely identifying lattice states;
- (e) verifying computation correctness through replay of deltas from initial state and comparison of recomputed fingerprints to claimed fingerprints.

Claim 2: A system for triangulation-free communication comprising:

- (a) a shared deterministic lattice with parameters (n, m, M) known to all participants;
- (b) encoding messages as lattice perturbations $P = \{(\text{row}_i, \text{col}_i, \delta_i)\}$;
- (c) applying perturbations to channel lattice state;
- (d) propagating messages through deterministic lattice evolution;
- (e) detecting messages through fingerprint matching without routing, addressing, or location information;
- (f) verifying transmission through replay consistency.

Claim 3: A method for accountable computation using digit-wise bounded arithmetic wherein:

- (a) all values exist in $\mathbb{Z}_m$ with no overflow or underflow;
- (b) arithmetic operations are performed digit-wise without carry propagation between digit positions;
- (c) zero is treated as a first-class value participating in all transforms;
- (d) equality is structural under transform constraints.

Dependent Claims

Claim 4: The method of Claim 1 wherein the transform matrix M has determinant invertible mod m , enabling reversible evolution.

Claim 5: The method of Claim 1 wherein proof of computation is generated as an observation receipt comprising timestamp, observer identifier, epoch number, lattice dimensions, fingerprint, and cryptographic signature.

Claim 6: The system of Claim 2 wherein multiple independent communication channels operate as parallel "lanes" without interference.

Claim 7: The system of Claim 2 wherein message reception is verified through:

- (a) perturbation hash matching;
- (b) replay verification from checkpoint; or
- (c) structural analysis of evolution trajectory.

Claim 8: The method of Claim 1 applied to AI inference accountability wherein model inputs are encoded to lattice, inference is represented as evolution, and receipts prove input-output relationships.

Claim 9: The method of Claim 1 applied to software execution logging wherein program state transitions are encoded as deltas enabling replay debugging.

Claim 10: The system of Claim 2 wherein communication privacy is achieved through location independence — participants are identified by perturbation signatures rather than addresses.

Part 6: Comparison to Prior Art

Aspect	Blockchain	CRDTs	TEE	DCRE
Consensus	Required (global)	None	Hardware trust	None
Proof Mechanism	PoW/PoS	Merge	Attestation	Replay
State Model	Append-only	Type-specific	Enclave	Bounded lattice
Verification	Expensive	Automatic	Hardware	Reconstruction
Communication	Routing	N/A	Routing	Triangulation-free

Novel Elements Not Found in Prior Art

1. **Digit-wise bounded arithmetic** with no carry propagation as computational foundation
 2. **Proof by replay** as sole verification mechanism
 3. **Triangulation-free communication** through lattice perturbation propagation
 4. **Universal lattice encoding** for any domain
 5. **Sparse delta evolution** with cryptographic binding
 6. **Bowling alley architecture** for parallel, independent communication channels
-

Part 7: Advantages

1. **No Trust Required:** Verification through reconstruction eliminates trusted authorities
2. **Compact Proofs:** Delta representation dramatically reduces storage requirements
3. **Universal Application:** Single primitive covers all domains
4. **Location Independence:** Communication without routing or triangulation

5. **Bounded Execution:** No numeric explosion or divergence possible
 6. **Deterministic Guarantees:** Same input always produces same output
 7. **Reversibility Option:** Invertible transforms enable backward analysis
 8. **Cycle Detection:** Finite state enables recurrence analysis
 9. **Parallel Operation:** Independent channels enable scalable communication
 10. **Privacy Preservation:** Fingerprint verification without content disclosure
-

Conclusion

DCRE provides a mathematically rigorous foundation for accountable computation and triangulation-free communication. The system's axiomatic specification ensures deterministic, bounded, and verifiable behavior across any domain. The bowling alley communication model enables location-independent message propagation verified through replay consistency rather than signal origin or path.

The combination of bounded arithmetic, lattice dynamics, delta evolution, proof by replay, and triangulation-free communication represents a novel computational primitive not anticipated by existing prior art in distributed systems, cryptography, or communication protocols.

Document prepared for USPTO Provisional Patent Application

© 2025 Bradley Clonan. All rights reserved.